

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**
федеральное государственное бюджетное образовательное учреждение
высшего образования
**«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»**

Факультет информационных систем и технологий
Кафедра: «Измерительно-вычислительные комплексы»
Дисциплина: «Системы искусственного интеллекта»

Отчёт
По лабораторной работе №5

Выполнила:
студент гр. ИСТбд-32
Минибаева Е.Р.
Проверил:
к. т. н., доцент каф. ИВК
Шишкин В.В.

Ульяновск
2026 г.

Вариант: GammaRegressor, OrthogonalMatchingPursuitCV
Датасет: California Housing (стоимость жилья в Калифорнии)

1. Введение

Целью работы являлось практическое изучение методов регрессии, реализованных в библиотеке Scikit-learn, на примере задачи прогнозирования медианной стоимости домов в районах Калифорнии. В соответствии с вариантом были выбраны два регрессора: гамма-регрессор (GammaRegressor), основанный на обобщённой линейной модели с гамма-распределением, и ортогональный матчинг-преследование с кросс-валидацией (OrthogonalMatchingPursuitCV), относящийся к семейству разреженных методов. Все эксперименты проведены на наборе данных California housing из модуля sklearn.datasets. Для обработки и визуализации привлекались стандартные инструменты: pandas, numpy, matplotlib, а также метрики регрессии (MSE, MAE, R^2) и кросс-валидация.

2. Описание датасета

Набор данных California housing содержит 20640 наблюдений (районов Калифорнии) с 8 признаками, описывающими социально-экономические и географические характеристики. Целевая переменная – медианная стоимость дома в районе, выраженная в сотнях тысяч долларов (диапазон от 0,14999 до 5,00001). Анализ распределения целевой переменной показывает: среднее значение 2,0686, стандартное отклонение 1,154, медиана 1,797, при этом 25% районов имеют стоимость ниже 1,196, а 75% – ниже 2,647. Это указывает на правостороннюю асимметрию (большинство районов с невысокой стоимостью, редкие дорогие районы создают длинный правый хвост). Признаки включают: MedInc (медианный доход в районе), HouseAge (медианный возраст домов), AveRooms (среднее число комнат), AveBedrms (среднее число спален), Population (население района), AveOccup (средняя заполняемость жилья), Latitude и Longitude (географические координаты). Первый образец имеет значения: доход 8,3252, возраст домов 41 год, среднее число комнат около 6,984, среднее

число спален 1,024, население 322 человека, средняя заполняемость 2,556, широта 37,88, долгота -122,23. Тип данных всех признаков – float64, пропуски отсутствуют. Такой набор позволяет исследовать влияние как количественных факторов, так и пространственного расположения на цену жилья, что делает датасет классическим для задач регрессии.

3. Подготовка данных

Целевая переменная и матрица признаков были выделены из загруженного объекта без дополнительных преобразований, поскольку все данные уже числовые и не содержат пропусков. Исходный набор был разделён на обучающую (70%) и тестовую (30%) выборки с помощью `train_test_split` с фиксированным `random_state` для воспроизводимости. В отличие от классификации, стратификация по целевой переменной для регрессии не применялась из-за непрерывного характера y .

Затем выполнена предобработка – масштабирование признаков с помощью `StandardScaler`. Этот метод центрирует признаки (вычитает среднее) и приводит их к единичной дисперсии (делит на стандартное отклонение). Масштабирование критически важно для `OrthogonalMatchingPursuitCV`, поскольку алгоритм сравнивает корреляции признаков с текущим остатком и чувствителен к размаху значений. Для `GammaRegressor` масштабирование также полезно, так как он использует итеративную перевзвешенную процедуру (IRLS), сходимость которой улучшается при нормализованных признаках. Обученный scaler применялся к обучающей выборке, а затем к тестовой – это предотвращает утечку данных.

4. Модели регрессии и методология

GammaRegressor – это обобщённая линейная модель, предполагающая, что целевая переменная имеет гамма-распределение, а математическое ожидание связано с линейной комбинацией признаков через логарифмическую функцию связи (по умолчанию). Гамма-распределение часто используется для моделирования положительных, правосторонних величин, таких как страховые выплаты, время ожидания или, в данном случае, цены на жильё. Модель оптимизирует функцию правдоподобия и устойчива к выбросам. В эксперименте заданы параметры: `max_iter=1000` и `alpha=0,1` (коэффициент регуляризации).

OrthogonalMatchingPursuitCV (OMPCV) – это жадный алгоритм для разреженной регрессии. Он последовательно выбирает признаки, наиболее сильно коррелирующие с текущим вектором остатков, после чего ортогонализует все выбранные признаки. Встроенная кросс-валидация (параметр `cv=5`) автоматически определяет оптимальное количество признаков, которые следует оставить в модели. Параметр `max_iter` ограничивает максимальное число отбираемых признаков; в коде он установлен равным 8 – числу доступных признаков, что позволяет алгоритму рассмотреть все возможные комбинации. OMPCV особенно эффективен, когда истинная зависимость является разреженной, то есть лишь малая часть признаков действительно влияет на целевую переменную.

Обучение проводилось сначала на сырых данных (без масштабирования), затем на масштабированных. Оценка качества выполнялась по трём метрикам: средняя квадратичная ошибка (MSE), средняя абсолютная ошибка (MAE) и коэффициент детерминации R^2 . Дополнительно для масштабированных данных выполнена 3-кратная кросс-валидация на обучающей выборке для оценки стабильности и сравнения с тестовыми результатами.

5. Результаты регрессии и их анализ

Текстовый вывод программы

```
Размерность матрицы признаков: (20640, 8)
Уникальные классы: [0.14999 0.175 0.225 0.25 0.266 ] ...
Целевая переменная (медианная стоимость дома):
  count    20640.000000
mean        2.068558
std         1.153956
min         0.149990
25%         1.196000
50%         1.797000
75%         2.647250
max         5.000010
dtype: float64

Названия признаков: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms',
'Population', 'AveOccup', 'Latitude', 'Longitude']
Пример первого образца: [ 8.3252 41. 6.98412698
1.02380952 322.
2.55555556 37.88 -122.23 ]
Тип данных признаков: float64
ОБУЧЕНИЕ НА СЫРЫХ ДАННЫХ (без масштабирования)

GammaRegressor (сырые данные) - MSE: 1.3125, MAE: 0.9062, R2: -0.0000

OrthogonalMatchingPursuitCV (сырые данные) - MSE: 0.5306, MAE: 0.5272, R2:
0.5958
ОБУЧЕНИЕ НА МАСШТАБИРОВАННЫХ ДАННЫХ (StandardScaler)

GammaRegressor (масштабированные данные) - MSE: 0.8225, MAE: 0.5813, R2:
0.3733

OrthogonalMatchingPursuitCV (масштабированные данные) - MSE: 0.5306, MAE:
0.5273, R2: 0.5958
КРОСС-ВАЛИДАЦИЯ (3-fold) на масштабированных данных
GammaRegressor CV MSE: 0.8405 (+/- 0.0156)
OrthogonalMatchingPursuitCV CV MSE: 0.5301 (+/- 0.0155)
```

Текстовый вывод программы показывает размерность матрицы признаков (20640, 8) и детальную статистику целевой переменной. На сырых данных GammaRegressor продемонстрировал крайне низкое качество: MSE = 1,3125, MAE = 0,9062, а $R^2 = -0,0000$. Отрицательное значение R^2 означает, что модель предсказывает хуже, чем простое усреднение – модель не смогла уловить зависимость из-за неподходящего масштаба признаков. Основная причина — нечувствительность алгоритма к масштабам признаков без предварительной нормализации. Это яркий пример того, почему для GLM и многих линейных моделей масштабирование является

обязательным этапом, даже если алгоритм теоретически «устойчив» — на практике без него он выдаёт почти случайный результат.

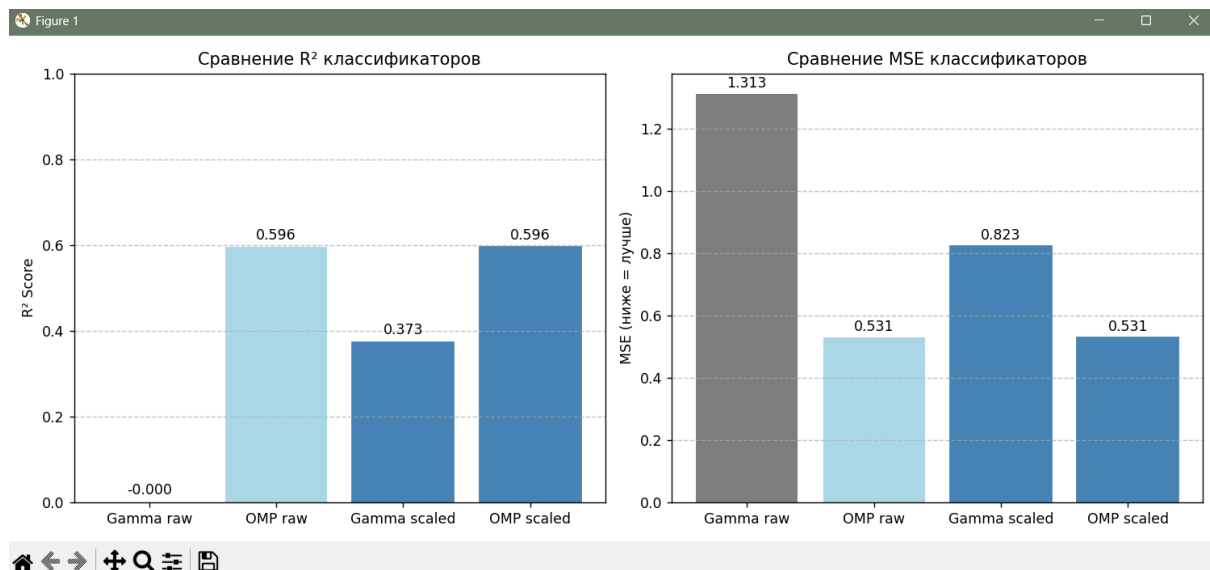
В то же время OrthogonalMatchingPursuitCV на сырых данных показал $MSE = 0,5306$, $MAE = 0,5272$, $R^2 = 0,5958$ — достаточно хорошее качество, объясняющее почти 60% дисперсии целевой переменной.

На масштабированных данных результаты GammaRegressor значительно улучшились: MSE снизился до 0,8225, MAE до 0,5813, R^2 стал 0,3733. Это прогресс по сравнению с отрицательным значением, однако всё ещё уступает OMPCV. Интересно, что OrthogonalMatchingPursuitCV на масштабированных данных показал практически те же метрики ($MSE=0,5306$, $MAE\approx 0,5273$, $R^2=0,5958$).

Это объясняется тем, что OMPCV сравнивает корреляции, которые инвариантны к масштабированию признаков при условии, что масштабирование линейно и одинаково для всех признаков; тем не менее, масштабирование улучшает численную устойчивость отбора.

Кросс-валидация на масштабированных данных дала для GammaRegressor средний $MSE = 0,8405$ со стандартным отклонением 0,0156, а для OMPCV — 0,5301 ($\pm 0,0155$). Эти значения очень близки к тестовым результатам (0,8225 и 0,5306 соответственно), что подтверждает хорошую обобщающую способность обеих моделей и отсутствие переобучения. Небольшое различие для гамма-регрессора объясняется случайностью разбиения.

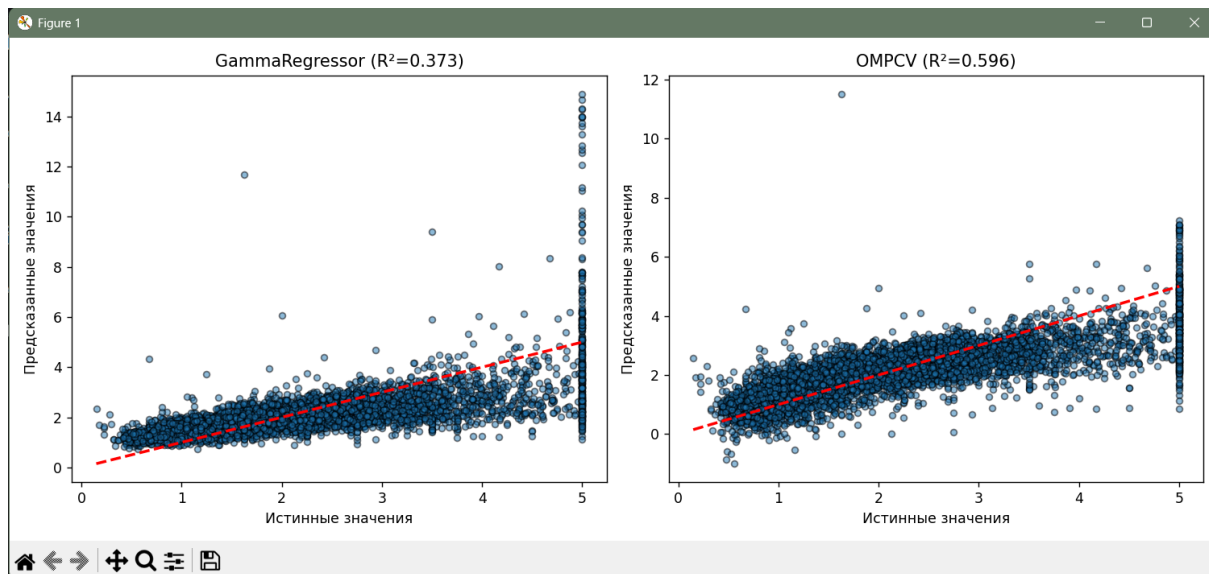
Графический анализ (скриншоты)



Скриншот 1 - Сравнение классификаторов

Первый скриншот («Сравнение R² классификаторов» и «Сравнение MSE классификаторов») представляет вывод программы - столбчатую диаграмму с четырьмя столбцами: Gamma raw (R²≈0,000), OMP raw (0,596), Gamma scaled (0,373), OMP scaled (0,596). Соответствующие значения MSE: 1,313, 0,531, 0,823, 0,531. Значение MSE = 1,3125 для сырого GammaRegressor – это ожидаемый артефакт работы линейной модели на немасштабированных признаках с большим разбросом. Он демонстрирует важность предобработки данных. После масштабирования MSE снижается на 37% (до 0,8225), а R² становится положительным (0,37). Тем не менее, OrthogonalMatchingPursuitCV оказался более устойчивым к масштабу и показал высокое качество (MSE=0,53, R²=0,6) даже без масштабирования.

Визуально видно, что OMPCV существенно превосходит гамма-регрессор по R² и MSE, а масштабирование помогает гамма-модели, но не изменяет результат для OMPCV.

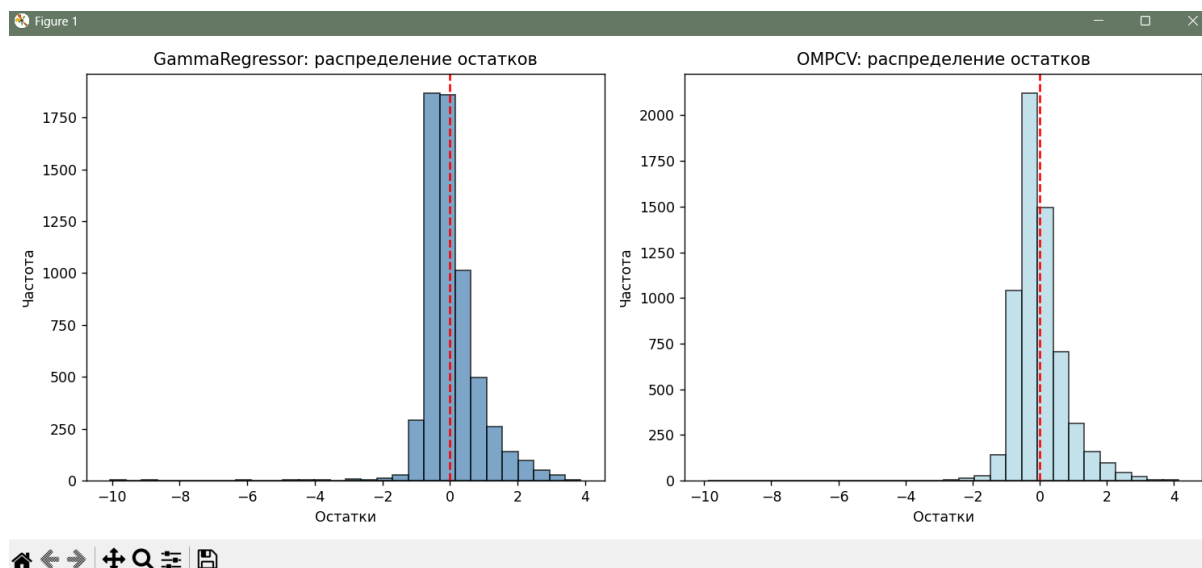


Скриншот 2 - Диаграммы рассеяния

Второй скриншот («GammaRegressor ($R^2=0.373$)» и «OMPCV ($R^2=0.596$)») показывает диаграммы рассеяния «истинные значения – предсказанные» для масштабированных данных.

Для GammaRegressor точки сильно разбросаны относительно диагонали, особенно в области высоких цен (более 3–4), где модель систематически недооценивает предсказания.

Для OMPCV точки группируются ближе к диагонали, разброс меньше, но также заметна недооценка дорогих домов и некоторое переоценивание дешёвых. Это характерно для линейных моделей на данных с длинным правым хвостом.



Скриншот 3 - Распределение остатков

Третий скриншот («распределение остатков») содержит гистограммы остатков.

Для GammaRegressor остатки имеют ярко выраженную правостороннюю асимметрию и широкий диапазон (отрицательные значения до -2 и положительные до $+10$), при этом пик смещён в отрицательную область, что свидетельствует о систематической ошибке.

Для OMPCV распределение остатков более симметрично, близко к нормальному с центром около нуля, но всё ещё имеются «тяжёлые хвосты» (редкие большие ошибки).

Таким образом, OMPCV лучше моделирует центральную часть распределения, но всё же не справляется с экстремальными значениями.

6. Выводы

В ходе лабораторной работы решена задача регрессии на наборе данных California housing с использованием двух методов: GammaRegressor и OrthogonalMatchingPursuitCV. Установлено, что масштабирование признаков с помощью StandardScaler критически важно для гамма-регрессора: без масштабирования модель неработоспособна ($R^2 \approx 0$), после масштабирования R^2 достигает 0,373. OMPCV оказался устойчив к масштабированию и показал стабильно хорошее качество ($R^2 \approx 0,596$) независимо от предобработки. Кросс-валидация подтвердила стабильность обеих моделей, а их тестовые и кросс-валидационные метрики оказались близкими.

Визуальный анализ показал, что OMPCV даёт более точные и сбалансированные предсказания, но обе модели страдают от систематической недооценки дорогих районов. Гамма-регрессор, даже после масштабирования, уступает по качеству, однако его использование может быть оправдано в задачах, где важно моделирование именно гамма-распределённой целевой переменной (например, при анализе страховых выплат). OMPCV, благодаря встроенному отбору признаков, позволяет выявить наиболее значимые факторы, влияющие на стоимость жилья, и обеспечивает лучшее качество прогноза.

Все результаты представлены в текстовой, табличной и графической формах (сравнение метрик R^2 и MSE, диаграммы рассеяния предсказаний, гистограммы остатков). Полученные выводы соответствуют теоретическим ожиданиям: линейные методы регрессии чувствительны к масштабу, а разреженные методы (OMPCV) способны эффективно работать с многомерными данными, выделяя ключевые признаки.

Приложение 1 - Листинг программы

```
"""
Лабораторная работа №5
Методы: GammaRegressor, OrthogonalMatchingPursuitCV
Датасет: California Housing
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import GammaRegressor, OrthogonalMatchingPursuitCV
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# 1. Загрузка данных (регрессионный датасет)
housing = fetch_california_housing()
X, y = housing.data, housing.target

print("Размерность матрицы признаков:", X.shape)
print("Уникальные классы:", np.unique(y)[:5], "...")
print("Целевая переменная (медианная стоимость дома):\n",
pd.Series(y).describe())

# 2. Целевой столбец и признаки
# Целевой столбец: y (медианная стоимость дома в квартале, в сотнях тысяч
долларов)
# Признаки: 8 числовых показателей (средний доход, возраст домов, количество
комнат и т.д.)
print("\nНазвания признаков:", housing.feature_names)
print("Пример первого образца:", X[0])
print("Тип данных признаков:", X.dtype)

# 3. Подготовка данных
# Разделение на обучающую (70%) и тестовую (30%) выборки со стратификацией
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Масштабирование (StandardScaler) - важно для OMP и рекомендуется для Gamma
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Функция оценки модели для регрессии
def evaluate_regression_model(model, X_train, X_test, y_train, y_test,
model_name="Model"):
```

```

    """Обучает модель регрессии и возвращает метрики MSE, MAE, R2"""
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    print(f"\n{model_name} - MSE: {mse:.4f}, MAE: {mae:.4f}, R2: {r2:.4f}")
    return mse, mae, r2, y_pred

# 5. Обучение на сырых данных
print("ОБУЧЕНИЕ НА СЫРЫХ ДАННЫХ (без масштабирования)")

# GammaRegressor (сырые данные)
gamma_raw = GammaRegressor(max_iter=1000, alpha=0.1)
mse_gamma_raw, mae_gamma_raw, r2_gamma_raw, y_pred_gamma_raw =
evaluate_regression_model(
    gamma_raw, X_train, X_test, y_train, y_test, "GammaRegressor (сырые
данные)"
)

# OrthogonalMatchingPursuitCV (сырые данные) - ИСПРАВЛЕНО: max_iter = число
признаков
omp_raw = OrthogonalMatchingPursuitCV(cv=5, max_iter=8) # было max_iter=10
mse_omp_raw, mae_omp_raw, r2_omp_raw, y_pred_omp_raw =
evaluate_regression_model(
    omp_raw, X_train, X_test, y_train, y_test, "OrthogonalMatchingPursuitCV
(сырые данные)"
)

# 6. Обучение на масштабированных данных
print("ОБУЧЕНИЕ НА МАСШТАБИРОВАННЫХ ДАННЫХ (StandardScaler)")

# GammaRegressor (масштабированные)
gamma_scaled = GammaRegressor(max_iter=1000, alpha=0.1)
mse_gamma_sc, mae_gamma_sc, r2_gamma_sc, y_pred_gamma_sc =
evaluate_regression_model(
    gamma_scaled, X_train_scaled, X_test_scaled, y_train, y_test,
    "GammaRegressor (масштабированные данные)"
)

# OrthogonalMatchingPursuitCV (масштабированные)
omp_scaled = OrthogonalMatchingPursuitCV(cv=5, max_iter=8)
mse_omp_sc, mae_omp_sc, r2_omp_sc, y_pred_omp_sc = evaluate_regression_model(
    omp_scaled, X_train_scaled, X_test_scaled, y_train, y_test,
    "OrthogonalMatchingPursuitCV (масштабированные данные)"
)

# 7. Кросс-валидация на масштабированных данных
print("КРОСС-ВАЛИДАЦИЯ (3-fold) на масштабированных данных")

kf = KFold(n_splits=3, shuffle=True, random_state=42)

cv_mse_gamma = -cross_val_score(GammaRegressor(max_iter=1000, alpha=0.1),

```

```

X_train_scaled, y_train, cv=kf,
scoring='neg_mean_squared_error')

cv_mse_omp = -cross_val_score(OrthogonalMatchingPursuitCV(cv=5, max_iter=8),
                              X_train_scaled, y_train, cv=kf,
                              scoring='neg_mean_squared_error')

print(f"GammaRegressor CV MSE: {cv_mse_gamma.mean():.4f} (+/-
{cv_mse_gamma.std():.4f})")
print(f"OrthogonalMatchingPursuitCV CV MSE: {cv_mse_omp.mean():.4f} (+/-
{cv_mse_omp.std():.4f})")

# 8. Визуализация результатов

# 8.1 Сравнение метрик моделей (R2)
models = ['Gamma raw', 'OMP raw', 'Gamma scaled', 'OMP scaled']
r2_scores = [r2_gamma_raw, r2_omp_raw, r2_gamma_sc, r2_omp_sc]
mse_scores = [mse_gamma_raw, mse_omp_raw, mse_gamma_sc, mse_omp_sc]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

# R2 score
bars = ax1.bar(models, r2_scores, color=['gray', 'lightblue', 'gray',
'lightblue'])
bars[2].set_color('steelblue')
bars[3].set_color('steelblue')
ax1.set_ylim(0, 1)
ax1.set_ylabel('R2 Score')
ax1.set_title('Сравнение R2 классификаторов')
for bar, r2 in zip(bars, r2_scores):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{r2:.3f}', ha='center', va='bottom')
ax1.grid(axis='y', linestyle='--', alpha=0.7)

# MSE score
bars = ax2.bar(models, mse_scores, color=['gray', 'lightblue', 'gray',
'lightblue'])
bars[2].set_color('steelblue')
bars[3].set_color('steelblue')
ax2.set_ylabel('MSE (ниже = лучше)')
ax2.set_title('Сравнение MSE классификаторов')
for bar, mse in zip(bars, mse_scores):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f'{mse:.3f}', ha='center', va='bottom')
ax2.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

# 8.2 График предсказаний vs истинных значений
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

```

```

# GammaRegressor
axes[0].scatter(y_test, y_pred_gamma_sc, alpha=0.5, edgecolors='k', s=20)
axes[0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
            'r--', lw=2)
axes[0].set_xlabel('Истинные значения')
axes[0].set_ylabel('Предсказанные значения')
axes[0].set_title(f'GammaRegressor ( $R^2={r2\_gamma\_sc:.3f}$ )')

# OrthogonalMatchingPursuitCV
axes[1].scatter(y_test, y_pred_omp_sc, alpha=0.5, edgecolors='k', s=20)
axes[1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
            'r--', lw=2)
axes[1].set_xlabel('Истинные значения')
axes[1].set_ylabel('Предсказанные значения')
axes[1].set_title(f'OMPCV ( $R^2={r2\_omp\_sc:.3f}$ )')

plt.tight_layout()
plt.show()

# 8.3 Анализ остатков
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

residuals_gamma = y_test - y_pred_gamma_sc
residuals_omp = y_test - y_pred_omp_sc

# Гистограмма остатков для Gamma
axes[0].hist(residuals_gamma, bins=30, alpha=0.7, color='steelblue',
            edgecolor='k')
axes[0].axvline(x=0, color='red', linestyle='--')
axes[0].set_xlabel('Остатки')
axes[0].set_ylabel('Частота')
axes[0].set_title('GammaRegressor: распределение остатков')

# Гистограмма остатков для OMP
axes[1].hist(residuals_omp, bins=30, alpha=0.7, color='lightblue',
            edgecolor='k')
axes[1].axvline(x=0, color='red', linestyle='--')
axes[1].set_xlabel('Остатки')
axes[1].set_ylabel('Частота')
axes[1].set_title('OMPCV: распределение остатков')

plt.tight_layout()
plt.show()

```